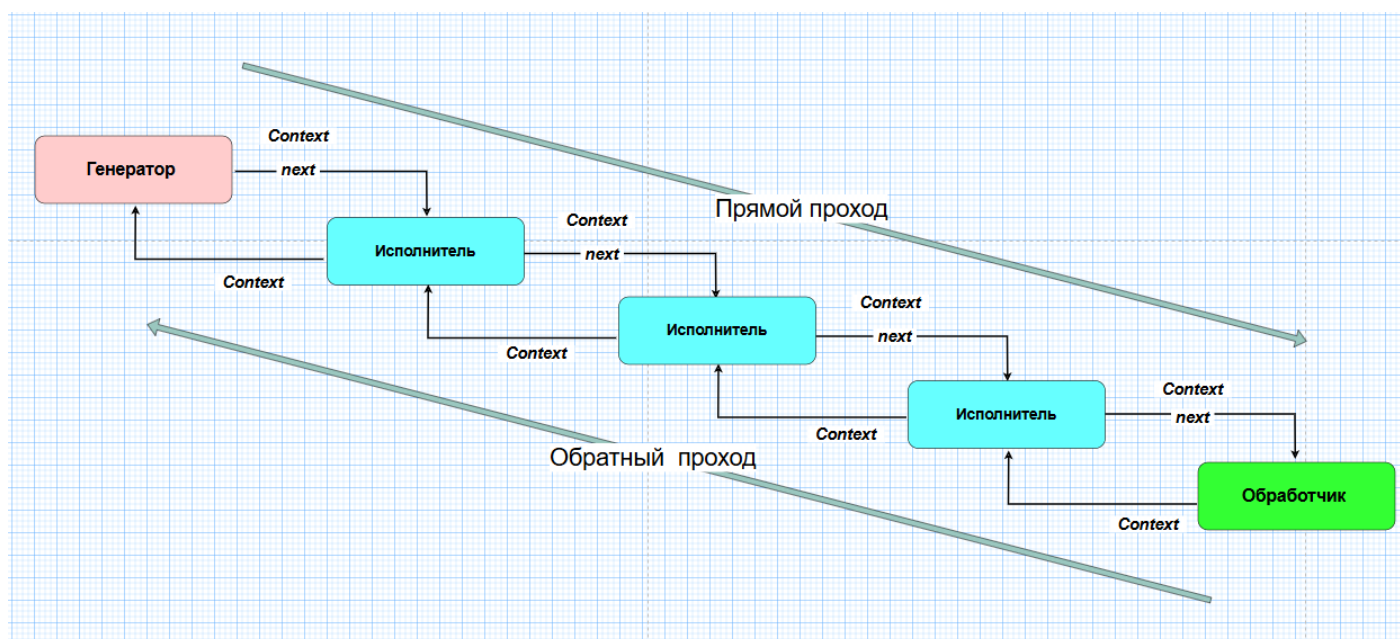


Chain of Responsibility (Responsibility Chain)

1. **RCH: Chain of Responsibility** – поведенческий паттерн проектирования, предназначенный для последовательной (по цепочке) обработке запроса обработчиками;
2. **RCH: Chain of Responsibility** – программный конвейер обработки данных.
3. **RCH: Middleware** – программный серверный компонент – реализация паттерна Chain of Responsibility.
4. **RCH:** чаще всего реализуется в виде последовательности вложенных вызовов функций (обработчиков – элементов цепочки).
5. **RCH:** каждый элемент цепи (конвейера) решает 3 задачи: 1) обработка запроса на прямом проходе запроса; 2) передача запроса следующему обработчику (может не передавать); 3) обработка запроса на обратном проходе запроса.
6. **RCH:** 3 типа элементов цепи: 1) стартовый – генератор запросов (генерирует запрос и отправляет его на обработку первому исполнителю); 2) промежуточный – исполнитель операции (принимает запрос, обрабатывает его, отправляет или не отправляет его следующему исполнителю) ; 3) конечный – обработчик запросов (принимает запрос и обрабатывает его).



7. RCH: интерфейсы

```
// IRC.dll
namespace IRC
{
    // T - Context - полезная нагрузка
    public interface IRCGenerator<T>           // генератор
    {
        Func<T, bool> Next { get; set; }       // функция перехода
        void Start(T context);               // первый в цепочке
    }

    public interface IRCHandler<T>           // обработчик
    {
        void LastUse(T context);             // последний в цепочке
    }

    public interface IResponsibilityChain<T> // исполнитель
    {
        IResponsibilityChain<T> Use(Func<T, Func<T, bool>, bool> step); // функция исполнителя
    }
}
```

8. RCH: реализация интерфейсов

```
// RC.dll
namespace RC
{
    public class RCGenerator : IRCGenerator<RCGenerator.Context>
    {
        public class Context           // полезная нагрузка
        {
            public int X { get; private set; }
            public int Y { get; private set; }
            public int Z { get; private set; }

            public Context(int x = 0, int y = 0, int z = 0) { this.X = x; this.Y = y; this.Z = z; }
            public Context() : this(0, 0, 0) { }
            public int sum() => this.Z += this.X + this.Y;
            public int sub() => this.Z += this.X - this.Y;
            public int mul() => this.Z += this.X * this.Y;
        }

        public Func<Context, bool> Next { get; set; } = null!; // <----- RChain;
        public void Start(Context context)                   // старт цепочки
        {
            Next?.Invoke(context);
            Console.WriteLine("Start: X = {0}, Y = {1}, Z = {2}", context.X, context.Y, context.Z);
        }
    }
}
```

```
public class RCHandler : IRCHandler<RCGenerator.Context>
{
    public void LastUse(RCGenerator.Context context)
    {
        Console.WriteLine("LastUse: X = {0}, Y = {1}, Z = {2}", context.X, context.Y, context.Z);
    }
}
```

```

public class RChain : IResponsibilityChain<RCGenerator.Context>
{
    private readonly IRCGenerator<RCGenerator.Context> generator;
    private readonly IRCHandler<RCGenerator.Context> handler;
    private readonly List<Func<RCGenerator.Context, Func<RCGenerator.Context, bool>, bool>> pipeline = new();

    public RChain(IRCGenerator<RCGenerator.Context> gen, IRCHandler<RCGenerator.Context> h )
    {
        generator = gen;
        handler = h;
        generator.Next = Execute;
    }

    public IResponsibilityChain<RCGenerator.Context> Use(Func<RCGenerator.Context, Func<RCGenerator.Context, bool>, bool> step)
    {
        pipeline.Add(step);
        return this;
    }

    private bool Execute(RCGenerator.Context ctx)
    {
        int index = 0;
        bool NextStep(RCGenerator.Context c)
        {
            bool rc = false;
            if (index >= pipeline.Count) handler.LastUse(c);
            else rc = pipeline[index++](c, NextStep); // рекурсия
            return rc;
        }
        return NextStep(ctx);
    }
}

```

9. RCH: пример

```

private static void Main(string[] args)
{
    var generator = new RCGenerator();
    var handler = new RCHandler();
    var rch = new RChain(generator, handler);

    rch.Use((context, next) =>
    {
        context.sum(); // прямой путь
        bool rc = next(context);
        return rc;
    })
    .Use((context, next) =>
    {
        context.sub(); // прямой путь
        bool rc = next(context);
        return rc;
    })
    .Use((context, next) =>
    {
        bool rc = next(context);
        context.mul(); // обратный путь
        return rc;
    });

    generator.Start(new RCGenerator.Context(3, 5));
    Console.WriteLine("-----");
    var c = new RCGenerator.Context(10, 5);
    generator.Start(c);
    generator.Start(new RCGenerator.Context(4, 6, c.Z));
}

```

```
LastUse: X = 3, Y = 5, Z = 6  
Start: X = 3, Y = 5, Z = 21  
-----  
LastUse: X = 10, Y = 5, Z = 20  
Start: X = 10, Y = 5, Z = 70  
LastUse: X = 4, Y = 6, Z = 78  
Start: X = 4, Y = 6, Z = 102
```

10. **КОНЕЦ**